

Laporan Praktikum dan Tugas Mandiri Machine Learning



Muhammad Riandana Pranatha - 0110224076

Teknik Informatika, STT Terpadu Nurul Fikri. Depok

0110224076@student.nurulfikri.ac.id

Abstrak

Praktikum ini dilakukan buat mempelajari cara membangun model Linear Regression dan ngecek performanya. Dataset day.csv dibaca pake pandas, terus beberapa kolom dipilih sebagai fitur (season, hum, windspeed, casual, registered) dan kolom temp sebagai target. Data dibagi jadi data latih dan data uji pake train_test_split (80% latih, 20% uji). Model Linear Regression dilatih pake data latih, terus dipake buat memprediksi nilai suhu (temp) pada data uji. Evaluasi model dilakukan pake Mean Squared Error (MSE) supaya bisa liat seberapa akurat prediksi model.

1. Menghubungkan Google Colab dengan Google Drive

kode ini dipake buat ngakses Google Drive dari Google Colab. perintah drive.mount('/content/drive') bikin folder /content/drive yang nyambung ke akun Drive kita, jadi file yang ada di Google Drive bisa langsung dibaca, disimpen, atau dipake di Colab.

```
5] 18 d from google.colab import drive
    drive.mount('/content/drive')

Mounted at /content/drive

5] 0 d path = "/content/drive/MyDrive/Praktikum//praktikum06/Data/"
```

1.2 Membaca Dataset CSV

kode ini digunakan buat membaca file socr.csv dan stunting_wasting_dataset.vsc dari folder yang udah ditentukan di path ke dalam DataFrame df.
df.head() dipake buat nampilin beberapa baris pertama dari data supaya bisa

diliat sekilas isi dan strukturnya.

```
[ ] ▶ # Membaca file csv menggunakan pandas
import pandas as pd

df = pd.read_csv(path + 'soccr.csv')
df.head()
```

▼ ...

	Index	Height(Inches)	Weight(Pounds)
0	1	65.78331	112.9925
1	2	71.51521	136.4873
2	3	69.39874	153.0269
3	4	68.21660	142.3354
4	5	67.78781	144.2971

```
1 ▶ # Membaca file csv menggunakan pandas
import pandas as pd

df = pd.read_csv(path + 'stunting_wasting_dataset.csv')
df.head()
```

▼ ...

	Jenis Kelamin	Umur (bulan)	Tinggi Badan (cm)	Berat Badan (kg)	Stunting	Wasting
0	Laki-laki	19	91.6	13.3	Tall	Risk of Overweight
1	Laki-laki	20	77.7	8.5	Stunted	Underweight
2	Laki-laki	10	79.0	10.3	Normal	Risk of Overweight
3	Perempuan	2	50.3	8.3	Severely Stunted	Risk of Overweight
4	Perempuan	5	56.4	10.9	Severely Stunted	Risk of Overweight

1.3 Statistik Deskriptif dan Pengecekan Missing Values

kode ini dipake buat ngeliat ringkasan statistik dari dataset df pake `describe()`, termasuk nilai rata-rata (mean), standar deviasi (std), nilai minimum, kuartil, dan maksimum tiap kolom.

Selain itu, `df.isnull().sum()` digunain buat ngecek apakah ada data yang kosong (missing values) di setiap kolom, biar bisa tau kalau ada kolom yang perlu dibersihkan atau diisi sebelum analisis lebih lanjut.

```
df.describe() .....# mean, std, min, quartiles, max
df.isnull().sum() .....# cek missing values
```

...	0
Jenis Kelamin	0
Umur (bulan)	0
Tinggi Badan (cm)	0
Berat Badan (kg)	0
Stunting	0
Wasting	0

dtype: int64

1.4 Membuat Kolom Baru

kode ini bikin kolom baru di DataFrame df dengan nama `tinggi_cm` dan `berat_kg` yang isinya sama dengan kolom asli `Tinggi Badan (cm)` dan `Berat Badan (kg)`.

`df.head()` dipake buat nampilin beberapa baris pertama supaya bisa dicek kalau kolom baru berhasil dibuat dan datanya sesuai.

```
df['tinggi_cm'] = df['Tinggi Badan (cm)']
df['berat_kg'] = df['Berat Badan (kg)']

df.head()
```

...	Jenis Kelamin	Umur (bulan)	Tinggi Badan (cm)	Berat Badan (kg)	Stunting	Wasting	tinggi_cm	berat_kg
0	Laki-laki	19	91.6	13.3	Tall	Risk of Overweight	91.6	13.3
1	Laki-laki	20	77.7	8.5	Stunted	Underweight	77.7	8.5
2	Laki-laki	10	79.0	10.3	Normal	Risk of Overweight	79.0	10.3
3	Perempuan	2	50.3	8.3	Severely Stunted	Risk of Overweight	50.3	8.3
4	Perempuan	5	56.4	10.9	Severely Stunted	Risk of Overweight	56.4	10.9

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

2. Tugas / Latihan Mandiri

Setelah kita mengakses Gdrive di colab dan membaca dataset nya, kita langsung mengeksekusi dataset nya yaitu day dan hour, langsung saja

2.1 Membaca File CSV hour.csv dan day.csv

kode ini digunakan buat membaca file hour dan day.csv dari folder yang udah ditentukan di path ke dalam DataFrame df.

df.head() dipake buat nampilin beberapa baris pertama dari data supaya bisa diliat sekilas isi dan strukturnya.

```
[24] # Membaca file csv menggunakan pandas
import pandas as pd

df = pd.read_csv(path + 'hour.csv')
df.head()
```

	instant	dteday	season	yr	mnth	hr	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	2011-01-01	1	0	1	0	0	6	0	1	0.24	0.2879	0.81	0.0	3	13	16
1	2	2011-01-01	1	0	1	1	0	6	0	1	0.22	0.2727	0.80	0.0	8	32	40
2	3	2011-01-01	1	0	1	2	0	6	0	1	0.22	0.2727	0.80	0.0	5	27	32
3	4	2011-01-01	1	0	1	3	0	6	0	1	0.24	0.2879	0.75	0.0	3	10	13
4	5	2011-01-01	1	0	1	4	0	6	0	1	0.24	0.2879	0.75	0.0	0	1	1

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

```
import pandas as pd

df_day = pd.read_csv(path + 'day.csv')
df_day.head()
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	2011-01-01	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	331	654	985
1	2	2011-01-02	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	131	670	801
2	3	2011-01-03	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	120	1229	1349
3	4	2011-01-04	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	108	1454	1562
4	5	2011-01-05	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	82	1518	1600

Langkah berikutnya: [Buat kode dengan df_day](#) [New interactive sheet](#)

2.2 Menentukan Fitur dan Target

kode ini buat milih kolom-kolom tertentu sebagai fitur (X) dan target (y) dari dataset.

- Untuk dataset day.csv, fitur yang dipakai adalah season, hum, windspeed, casual, dan registered, sedangkan targetnya adalah temp.
- Untuk dataset hour.csv, fitur yang dipilih cuma season, hum, dan windspeed, dengan target tetap temp.

X.head() dan y.head() dipakai buat nampilin beberapa baris pertama dari fitur dan target supaya bisa dicek sekilas datanya.

```
import pandas as pd

df_day = pd.read_csv(path + 'day.csv')

X = df_day[['season', 'hum', 'windspeed', 'casual', 'registered']]

y = df_day[['temp']]

X.head(), y.head()
```

	season	hum	windspeed	casual	registered
0	1	0.805833	0.160446	331	654
1	1	0.696087	0.248539	131	670
2	1	0.437273	0.248309	120	1229
3	1	0.590435	0.160296	108	1454
4	1	0.436957	0.186900	82	1518,

0	0.344167
1	0.363478
2	0.196364
3	0.200000
4	0.226957

Name: temp, dtype: float64)

```
import pandas as pd # import sendiri supaya blok ini jalan sendiri
df_hour = pd.read_csv(path + 'hour.csv')

X = df_hour[['season', 'hum', 'windspeed']]

y = df_hour[['temp']]

X.head(), y.head()
```

	season	hum	windspeed
0	1	0.81	0.0
1	1	0.80	0.0
2	1	0.80	0.0
3	1	0.75	0.0
4	1	0.75	0.0,

0	0.24
1	0.22
2	0.22
3	0.24
4	0.24

Name: temp, dtype: float64)

2.2 Pemisahan Data Latih dan Data Uji

kode ini digunakan untuk membagi dataset menjadi data latih dan data uji menggunakan fungsi `train_test_split` dari `sklearn`.

- `test_size=0.2` artinya 20% data dijadikan data uji dan 80% sisanya jadi data latih.
- `random_state=42` bikin pembagian data konsisten setiap kali dijalankan.

Hasilnya, `X_train` dan `y_train` bakal dipakai untuk melatih model, sedangkan `X_test` dan `y_test` dipakai untuk mengecek performa model. `X_train.shape` dan `X_test.shape` nunjukkan jumlah baris dan kolom dari masing-masing dataset setelah dipisah.

```
import pandas as pd
from sklearn.model_selection import train_test_split

df_day = pd.read_csv(path + 'day.csv')
X = df_day[['season', 'hum', 'windspeed', 'casual', 'registered']]
y = df_day['temp']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape

... ((584, 5), (147, 5))
```

```
import pandas as pd
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape

((13903, 3), (3476, 3))
```

2.3 Pelatihan Model Linear Regression

kode ini digunakan untuk membuat dan melatih model Linear Regression.

- `LinearRegression()` bikin objek model baru.
 - `model.fit(X_train, y_train)` ngebuat model belajar dari data latih supaya bisa ngerti pola hubungan antara fitur (`X_train`) dan target (`y_train`).
- Setelah tahap ini, model siap dipake buat memprediksi nilai target dari data uji.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df_day = pd.read_csv(path + 'day.csv')
X = df_day[['season', 'hum', 'windspeed', 'casual', 'registered']]
y = df_day['temp']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?
LinearRegression()


```
13] 0d ▶ import pandas as pd
      from sklearn.linear_model import LinearRegression

      model = LinearRegression()
      model.fit(X_train, y_train)

      ...
      ▼ LinearRegression ⓘ ?
      LinearRegression()
```

2.2 Prediksi Nilai Target dengan Model Linear Regression

kode ini dipake buat memprediksi nilai target (temp) pada data uji (X_test) pake model Linear Regression yang udah dilatih.

- `model.predict(X_test)` ngasih hasil prediksi dan disimpan di `y_pred`.
- `y_pred[:5]` nampilin 5 hasil prediksi pertama buat ngecek sekilas keluaran model.

```
2] 0d ▶ import pandas as pd

      df_day = pd.read_csv(path + 'day.csv')
      X = df_day[['season', 'hum', 'windspeed', 'casual', 'registered']]
      y = df_day['temp']

      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
      model = LinearRegression()
      model.fit(X_train, y_train)

      y_pred = model.predict(X_test)
      y_pred[:5]

      ... array([0.61735545, 0.26266085, 0.45051657, 0.65087224, 0.64133824])
```

```
]
0d ▶ import pandas as pd

    y_pred = model.predict(X_test)
    y_pred[:5]

... array([0.57049436, 0.43831811, 0.57589914, 0.47823417, 0.41561611])
```

2.2 Evaluasi Model Linear Regression

kode ini dipake buat ngukur performa model Linear Regression pake Mean Squared Error (MSE).

- `model.predict(X_test)` dipake buat dapetin prediksi dari data uji.
- `mean_squared_error(y_test, y_pred)` ngitung rata-rata kuadrat selisih antara nilai aktual (`y_test`) sama nilai prediksi (`y_pred`).
- `print()` nampilin nilai MSE, makin kecil MSE berarti prediksi model makin akurat.

```
5] ▶ import pandas as pd
    from sklearn.metrics import mean_squared_error

    mse = mean_squared_error(y_test, y_pred)
    print("Mean Squared Error:", mse)

✓ ... Mean Squared Error: 0.03224374142081836
```

```
23] import pandas as pd
0d from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df_day = pd.read_csv(path + 'day.csv')
X = df_day[['season', 'hum', 'windspeed', 'casual', 'registered']]
y = df_day['temp']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

... Mean Squared Error: 0.01771414338968397

Kesimpulan

Dari praktikum ini bisa disimpulkan kalau Linear Regression bisa dipake buat memprediksi nilai target berdasarkan beberapa fitur. Dengan ngecek MSE, bisa diliat seberapa akurat model, dimana nilai MSE yang makin kecil nunjuki prediksi yang lebih baik. Praktikum ini nunjuki langkah dasar mulai dari baca data, milih fitur dan target, bagi data, latihan model, sampe evaluasi performa model.

Berikut Link Github dan Google Drive:

Github : <https://github.com/Sakiaihara12/Machine-Learning.git>

Drive:

https://drive.google.com/drive/folders/1z4s6VxDRpkNs1RD_qpknpsDESOR8IMQ2?usp=s_haring